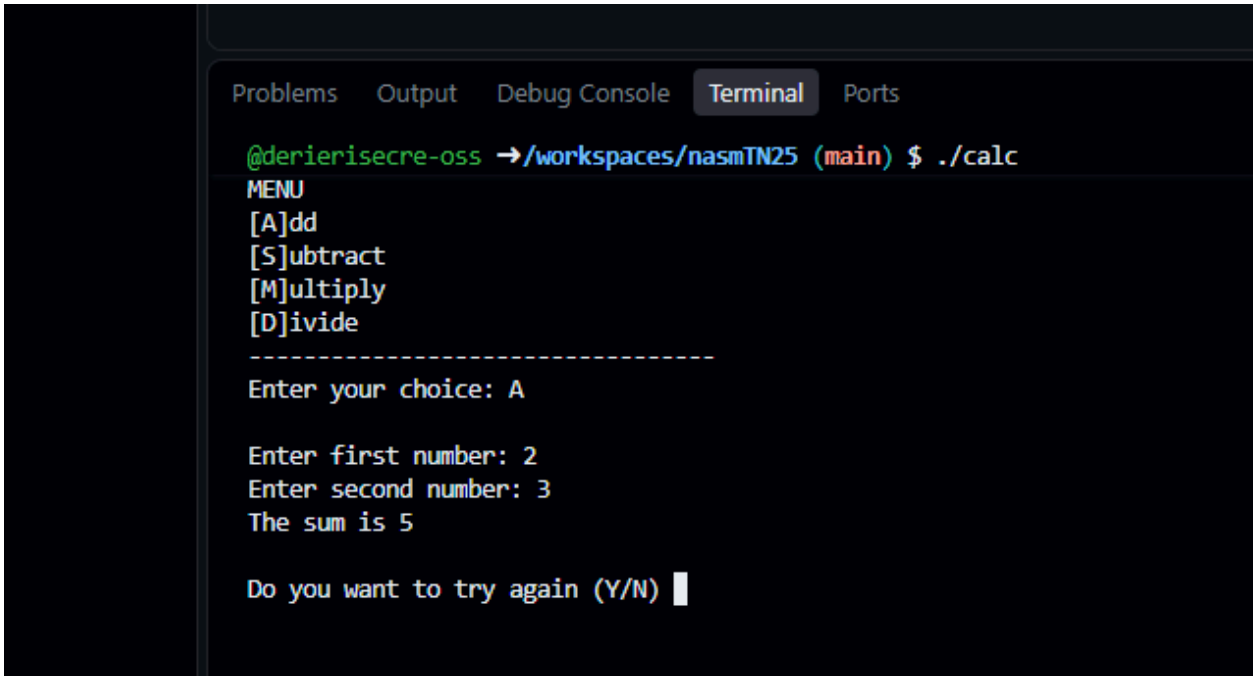
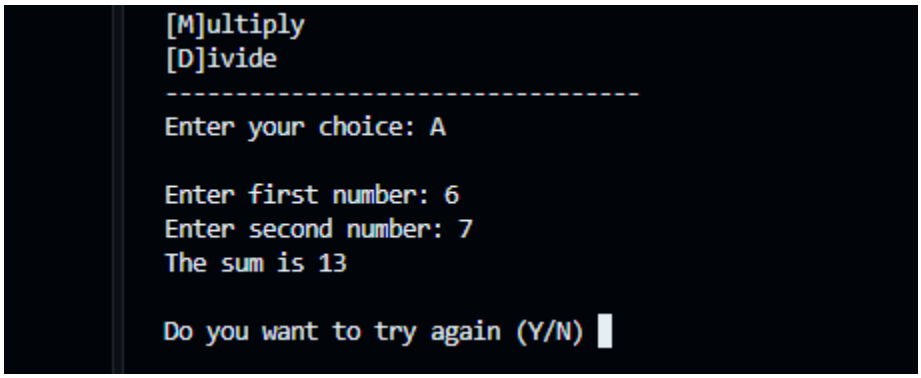
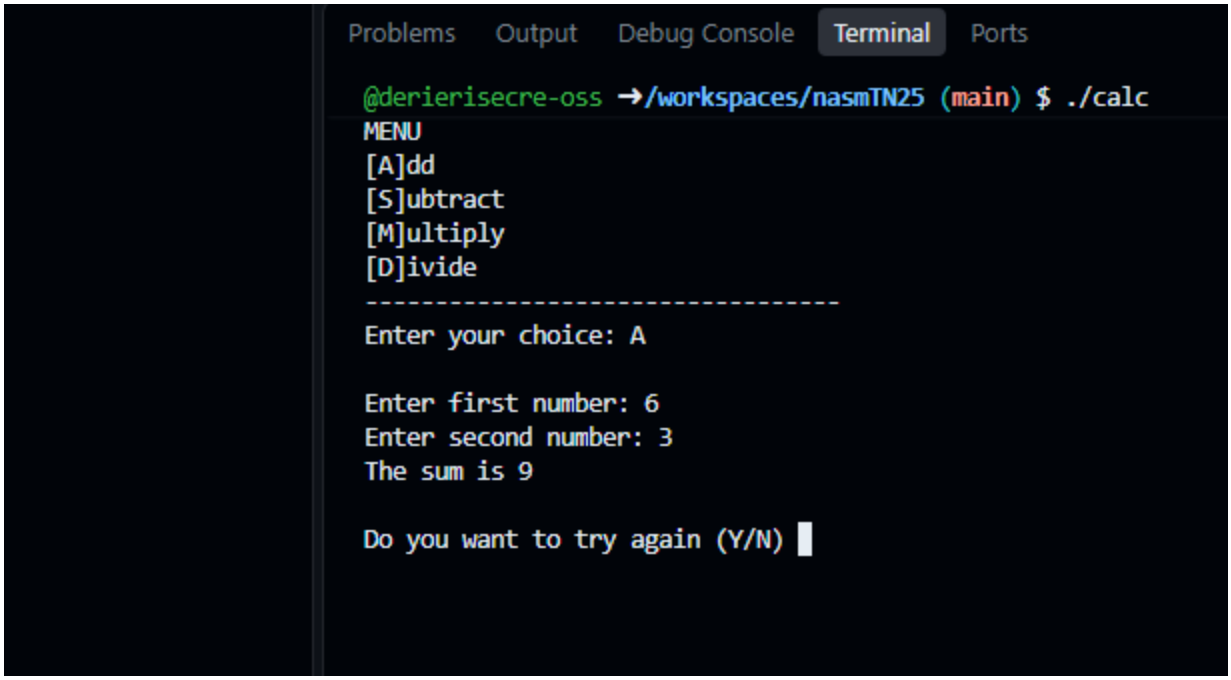

Problems
Output
Debug Console
Terminal
Ports
@derierisecre-oss ➜/workspaces/nasmTN25 (main) $ ./calc
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide

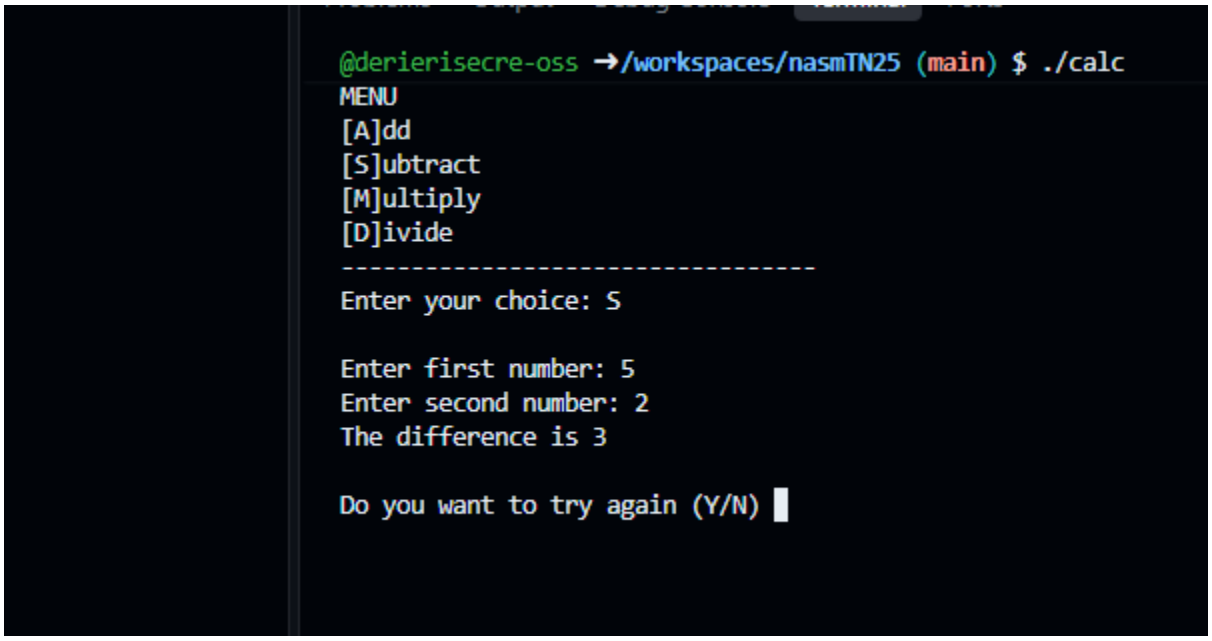
Enter your choice: A
Enter first number: 2
Enter second number: 3
The sum is 5
Do you want to try again (Y/N)


[M]ultiply
[D]ivide

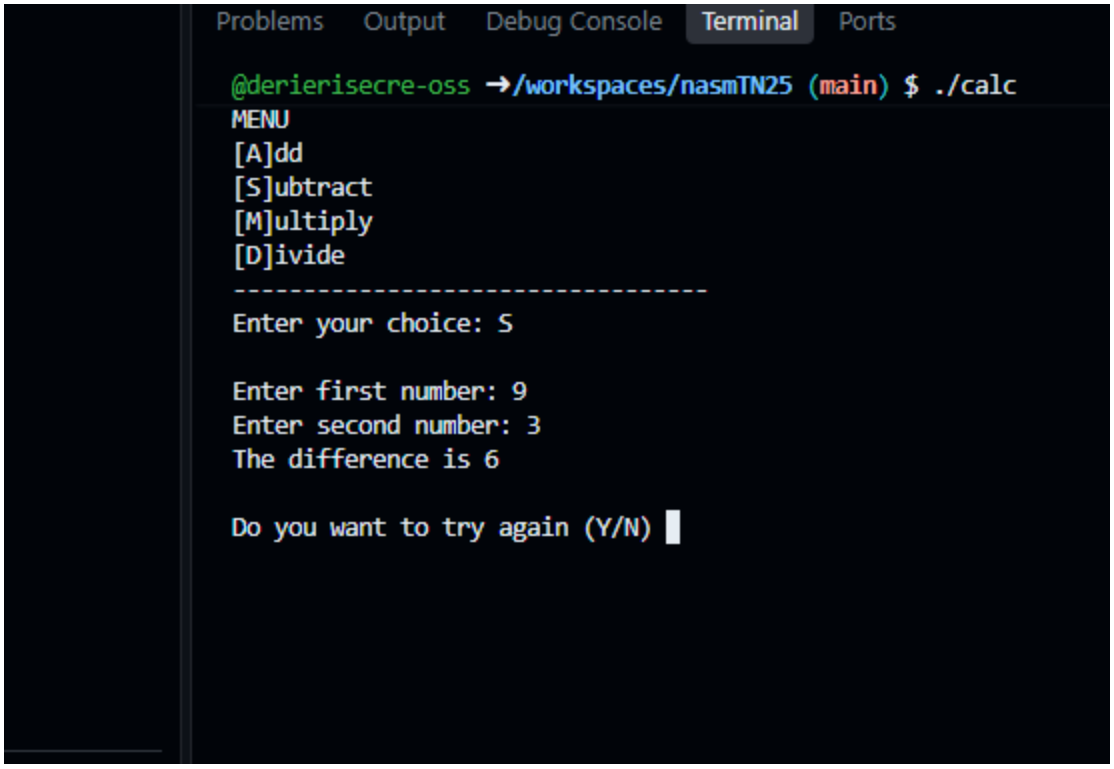
Enter your choice: A
Enter first number: 6
Enter second number: 7
The sum is 13
Do you want to try again (Y/N)


Problems
Output
Debug Console
Terminal
Ports
@derierisecre-oss ➜/workspaces/nasmTN25 (main) $ ./calc
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide

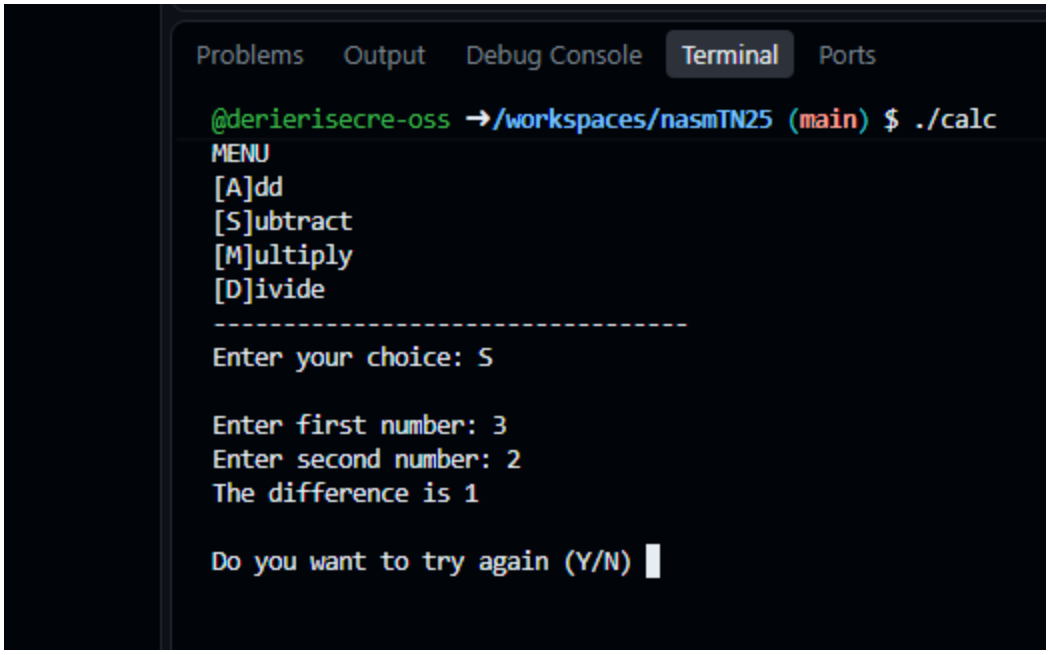
Enter your choice: A
Enter first number: 6
Enter second number: 3
The sum is 9
Do you want to try again (Y/N)


@derierisecre-oss ➜/workspaces/nasmTN25 (main) $ ./calc
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide

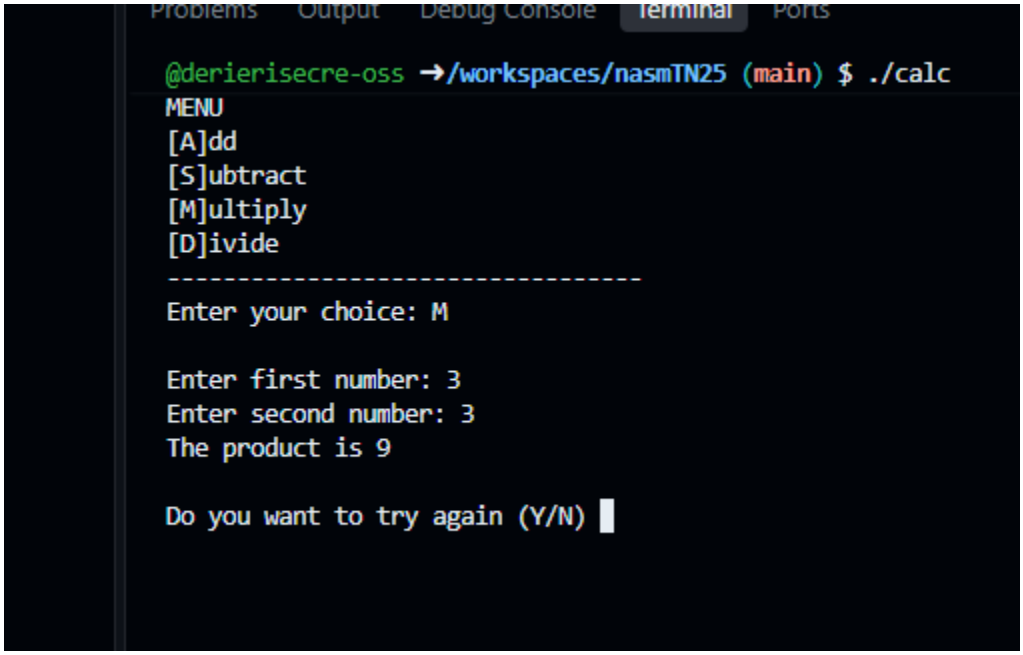
Enter your choice: S
Enter first number: 5
Enter second number: 2
The difference is 3
Do you want to try again (Y/N)


Problems
Output
Debug Console
Terminal
Ports
@derierisecre-oss ➜/workspaces/nasmTN25 (main) $ ./calc
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide

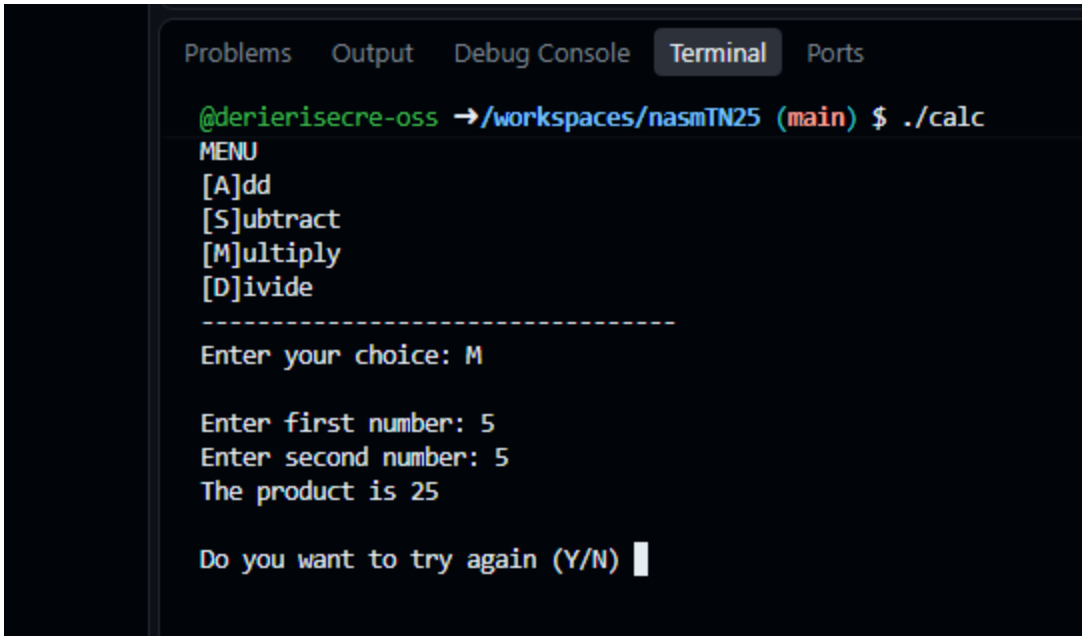
Enter your choice: S
Enter first number: 9
Enter second number: 3
The difference is 6
Do you want to try again (Y/N)


Problems
Output
Debug Console
Terminal
Ports
@derierisecre-oss →/workspaces/nasmTN25 (main) $ ./calc
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide

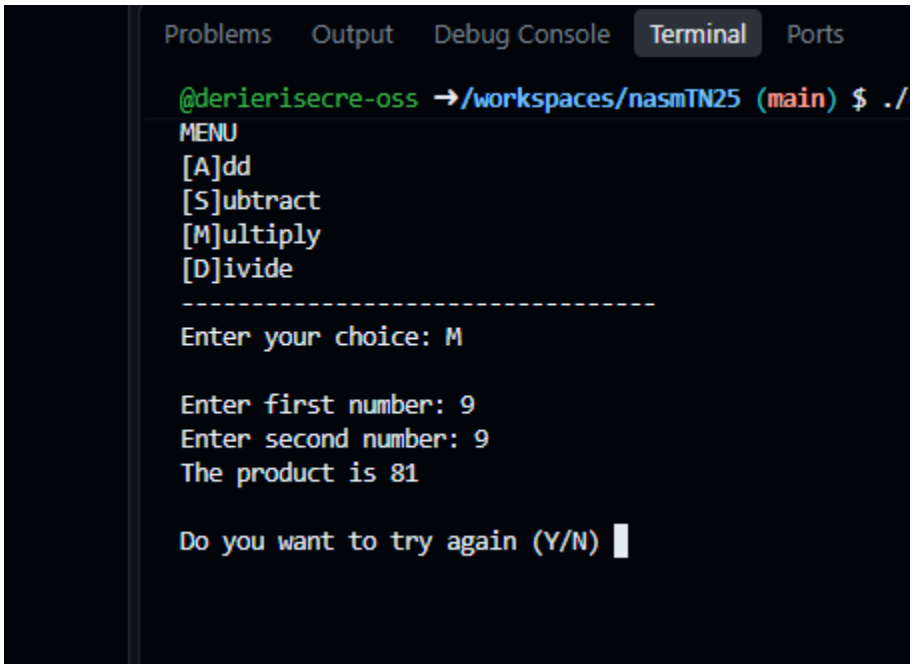
Enter your choice: S
Enter first number: 3
Enter second number: 2
The difference is 1
Do you want to try again (Y/N)


Problems
Output
Debug Console
Terminal
Ports
@derierisecre-oss →/workspaces/nasmTN25 (main) $ ./calc
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide

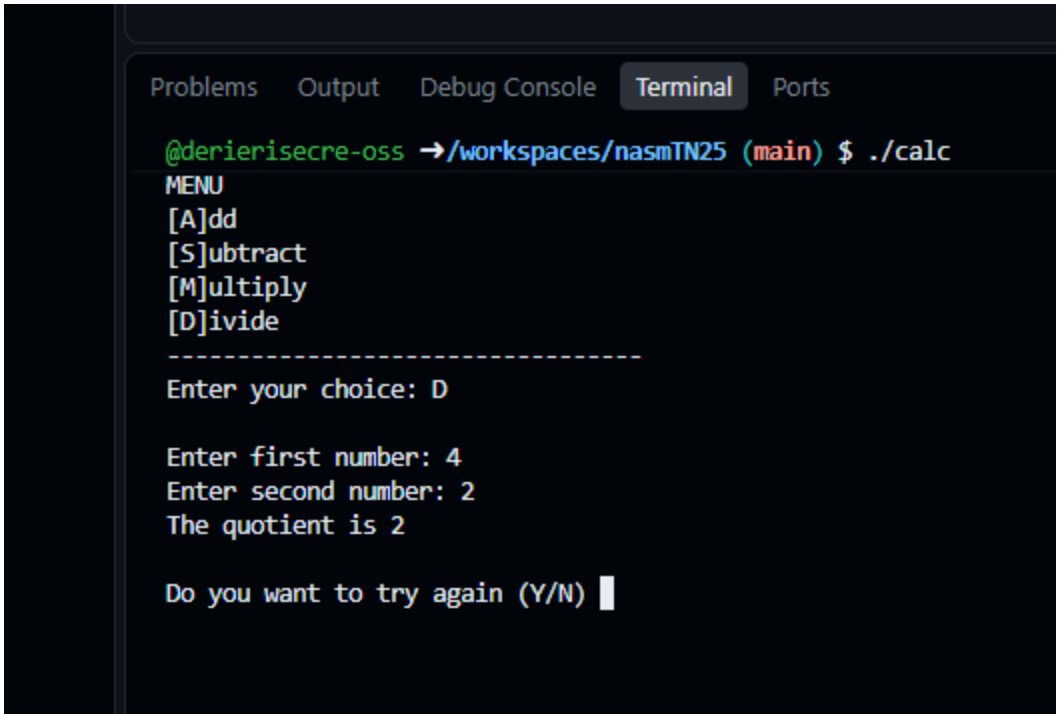
Enter your choice: M
Enter first number: 3
Enter second number: 3
The product is 9
Do you want to try again (Y/N)


Problems
Output
Debug Console
Terminal
Ports
@derierisecre-oss ➜/workspaces/nasmTN25 (main) $ ./calc
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide

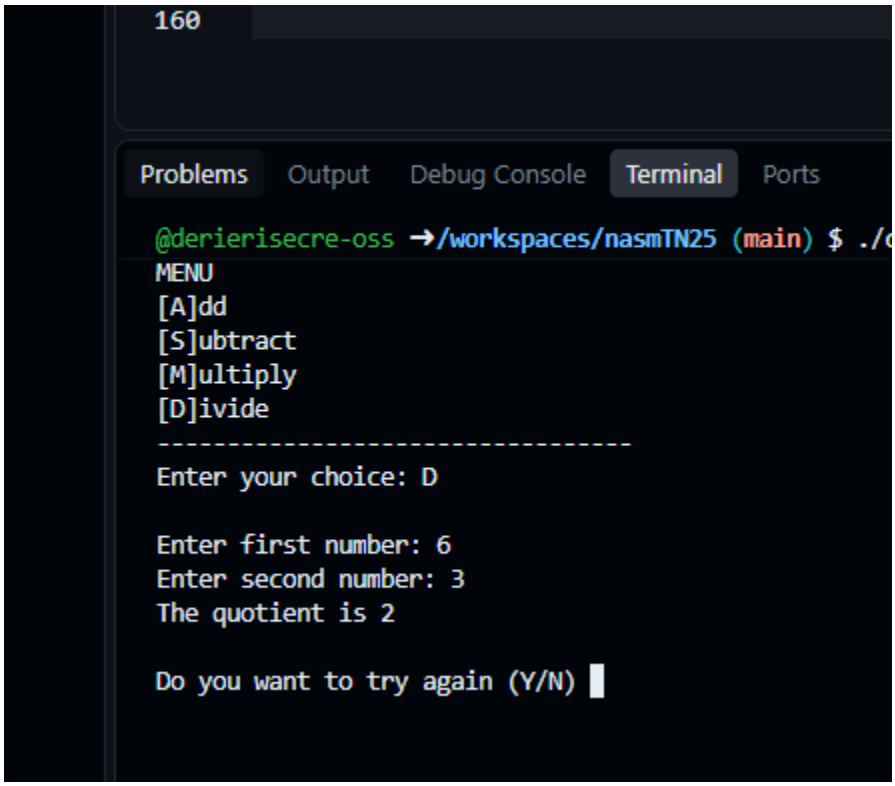
Enter your choice: M
Enter first number: 5
Enter second number: 5
The product is 25
Do you want to try again (Y/N)


Problems
Output
Debug Console
Terminal
Ports
@derierisecre-oss ➜/workspaces/nasmTN25 (main) $ ./
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide

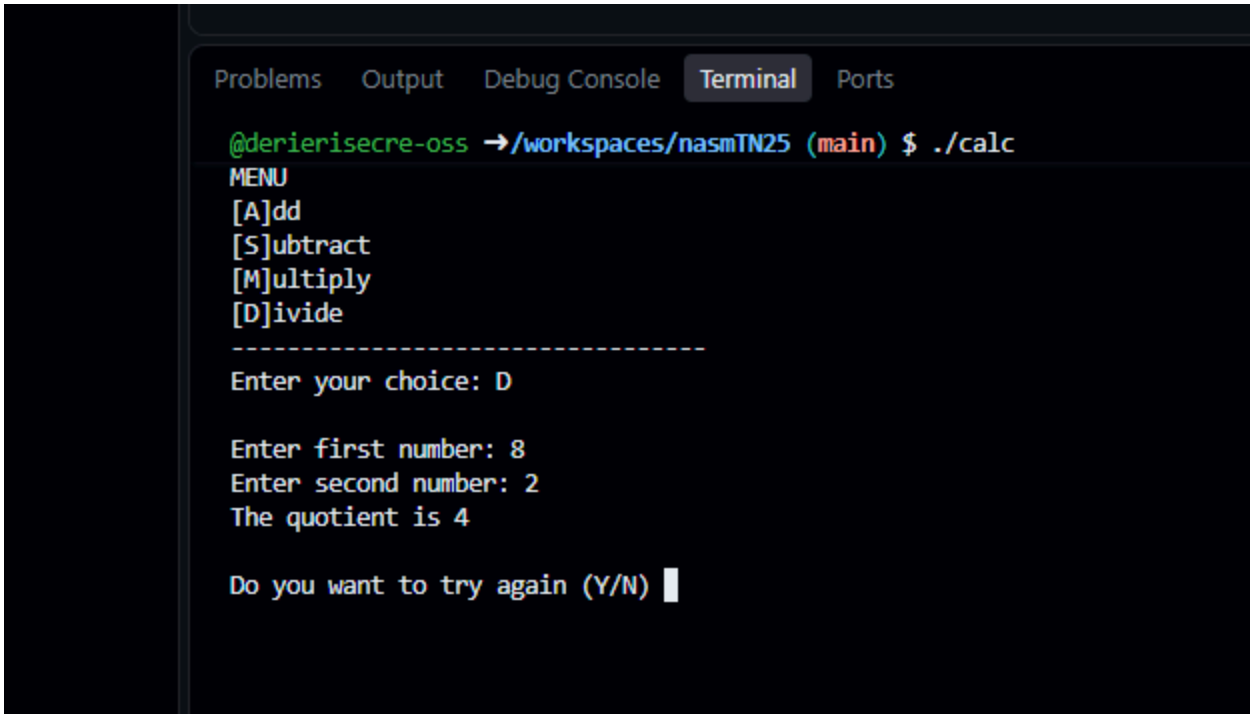
Enter your choice: M
Enter first number: 9
Enter second number: 9
The product is 81
Do you want to try again (Y/N)


Problems
Output
Debug Console
Terminal
Ports
@derierisecre-oss →/workspaces/nasmTN25 (main) $ ./calc
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide

Enter your choice: D
Enter first number: 4
Enter second number: 2
The quotient is 2
Do you want to try again (Y/N)


160
Problems
Output
Debug Console
Terminal
Ports
@derierisecre-oss →/workspaces/nasmTN25 (main) $ ./
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide

Enter your choice: D
Enter first number: 6
Enter second number: 3
The quotient is 2
Do you want to try again (Y/N)



1. Program layout

The file is split into sections:

- **Macros:** shortcuts that the assembler expands into full instructions before building the program.
- **`.data`:** the fixed text the program displays (menu, prompts, result labels). Each `equ $ - label` line calculates that text's length.
- **`.bss`:** empty storage for what the user types (`choice`, `num1`, `num2`, `answer`) and for the result text.
- **`.text`:** the actual code, starting at `_start`.

2. The macros

- **`print text, length`** displays text on the screen using `sys_write` (`eax = 4`, `ebx = 1` for the screen), then `int 0x80` asks Linux to do it.
- **`read place`** waits for typing using `sys_read` (`eax = 3`, `ebx = 0` for the keyboard) and stores the typed characters in `place`.
- **`load_numbers`** takes the two typed characters and turns them into real numbers. The character `'5'` is stored as the code 53, so subtracting `'0'` (48) gives the number 5. The numbers end up in `eax` and `ebx`.

3. Main flow

1. **Show the menu.** The screen is cleared by printing the ANSI codes `ESC[2J ESC[H`, then the menu and the “Enter your choice” prompt are printed.
2. **Read the inputs.** The program reads the menu letter, then the first number, then the second number.
3. **Pick an operation.** `cmp` checks the letter against `A`, `S`, `M` and `D`. `je` (“jump if equal”) sends the program to the matching procedure. Any other letter skips straight to “try again”.
4. **Run the operation.** Each one prints its label, calls `load_numbers`, then calculates:
    - Add uses `add eax, ebx`.
    - Subtract uses `sub eax, ebx`.
    - Multiply uses `imul eax, ebx`.
    - Divide first sets `edx` to 0, then `div ebx` leaves the quotient in `eax`.
5. **Show the answer.** `show_result` prints the number in `eax`.
6. **Try again?** `Y` jumps back to `show_menu`, which clears the screen and repeats. Anything else ends the program with `sys_exit` (`eax = 1`).

4. Printing a two-digit answer

The CPU gives a number, but the screen needs text characters. `show_result` handles this:

1. It divides the answer by 10 with `div`. The quotient (`eax`) is the tens digit and the remainder (`edx`) is the ones digit.
2. It adds `'0'` to each digit to turn the numbers back into characters, then stores them with a new line.
3. If the tens digit is 0 (like `05`), it skips that character so the screen shows `5`.
4. It writes the text with `sys_write`.

For example, 9 × 9 gives 81. Dividing by 10 gives tens 8 and ones 1, which are converted to `'8'` and `'1'` and printed as `81`.

5. Example run (Add, 2 + 3)

1. The user types `A`, `2` and `3`.
2. `cmp` matches `A` and jumps to `do_add`.
3. The label “The sum is “ is printed.
4. `load_numbers` puts 2 in `eax` and 3 in `ebx`.
5. `add eax, ebx` makes `eax` equal 5.
6. `show_result` splits 5 into tens 0 and ones 5, skips the leading zero, and prints `5`.
7. “Do you want to try again (Y/N)” appears. `Y` loops back to the menu and `N` exits.

6. Limitations

- The numbers entered are single digits (0–9), but answers can reach two digits.
- Letters must be capitals.
- The first number must be at least the second for subtraction, since negative answers aren’t handled.
- The second number can’t be 0 for division, because the CPU would crash.